**FORMATO PARA EL DESARROLLO DE COMPONENTE FORMATIVO**

| | |
|---|---|
| **PROGRAMA DE FORMACIÓN** | Análisis y desarrollo de software |

| | | | |
|---|---|---|---|
| **Competencia** | 220501096 - Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo | **Resultados de aprendizaje** | 22050109601 - Planear actividades de construcción del software de acuerdo con el diseño establecido. |

| | |
|---|---|
| **Número del componente formativo** | N/A |
| **Nombre del componente formativo** | Trabajando con Git y GitHub |
| **Breve descripción** | En este instructivo se guía al aprendiz en el proceso de colaboración en proyectos de GitHub mediante el uso de forks. A través de un ejemplo práctico, se muestra cómo cargar un programa propio en GitHub, permitir que otro usuario realice un fork del repositorio, y cómo este colaborador puede clonar, modificar y aplicar mejoras al código siguiendo el estándar PEP 8 de estilo en Python. Finalmente, se enseña cómo enviar los cambios mediante un pull request para integrarlos al proyecto original, fomentando el trabajo colaborativo y las buenas prácticas de programación. |
| **Palabras clave** | Git, GitHub, Fork, Trabajo colaborativo, Merge |
| **Área ocupacional** | Informática |

## Contenido

1. Introducción ..... 3

2. Instalación de Git ..... 3

3. Ejercicio para pruebas ..... 12

4. Repositorio de GitHub ..... 14

5. Trabajo colaborativo con fork ..... 22

6. Referencias ..... 35

# Trabajando con Git y GitHub

## 1. Introducción

Git es un sistema de control de versiones que permite registrar los cambios realizados en un proyecto, facilitando el trabajo colaborativo y el seguimiento del historial de modificaciones.

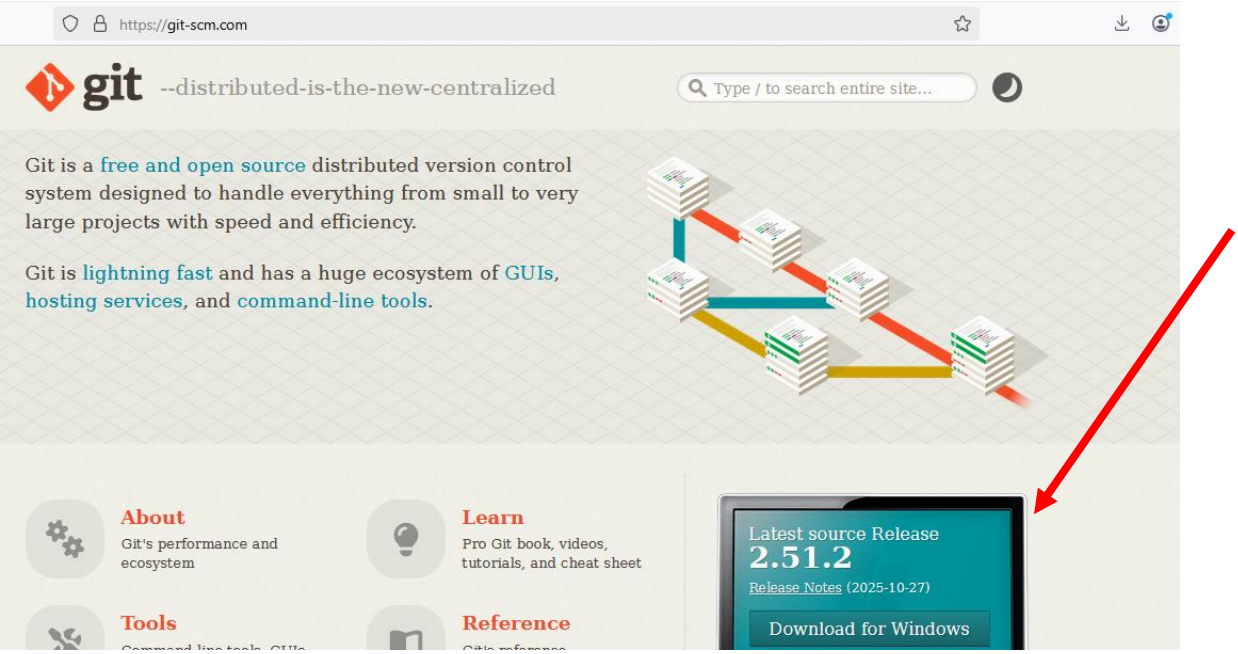
GitHub es una plataforma en la nube que utiliza Git y permite almacenar, compartir y gestionar proyectos de software, ofreciendo herramientas para la colaboración entre desarrolladores.

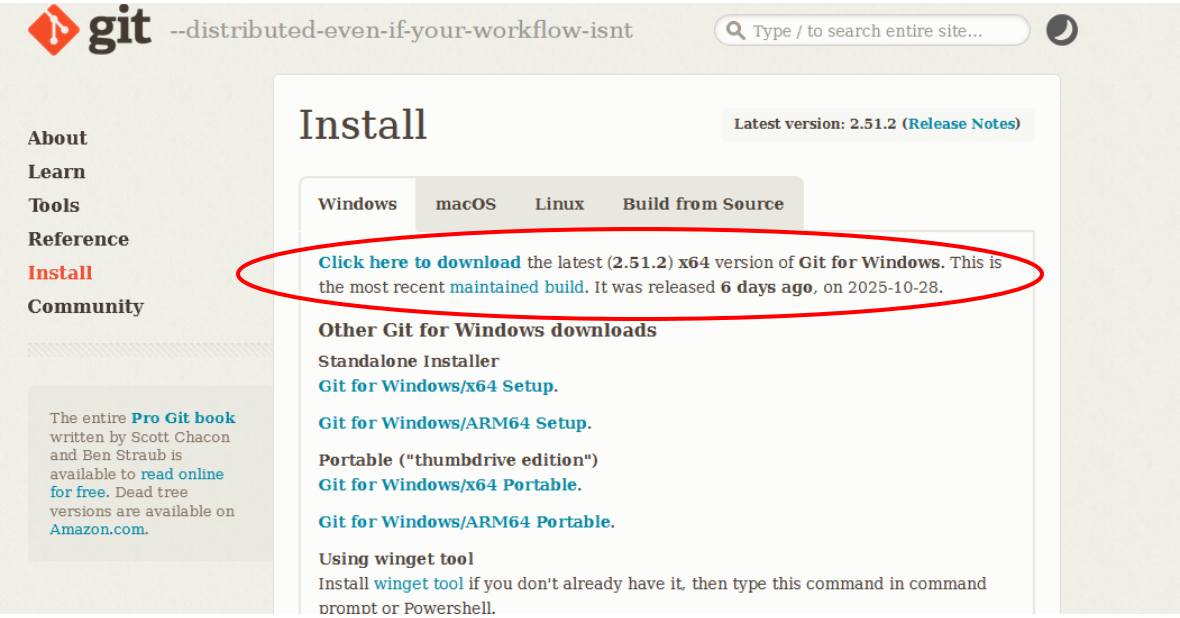
Un fork es una copia de un repositorio de GitHub que se crea en tu propia cuenta, permitiéndote modificar el proyecto original sin afectar la versión principal, y luego proponer mejoras o contribuciones mediante un pullrequest.

En síntesis, Git gestiona los cambios, GitHub aloja los proyectos y el fork permite colaborar y contribuir a proyectos de otros de forma segura.

## 2. Instalación de Git

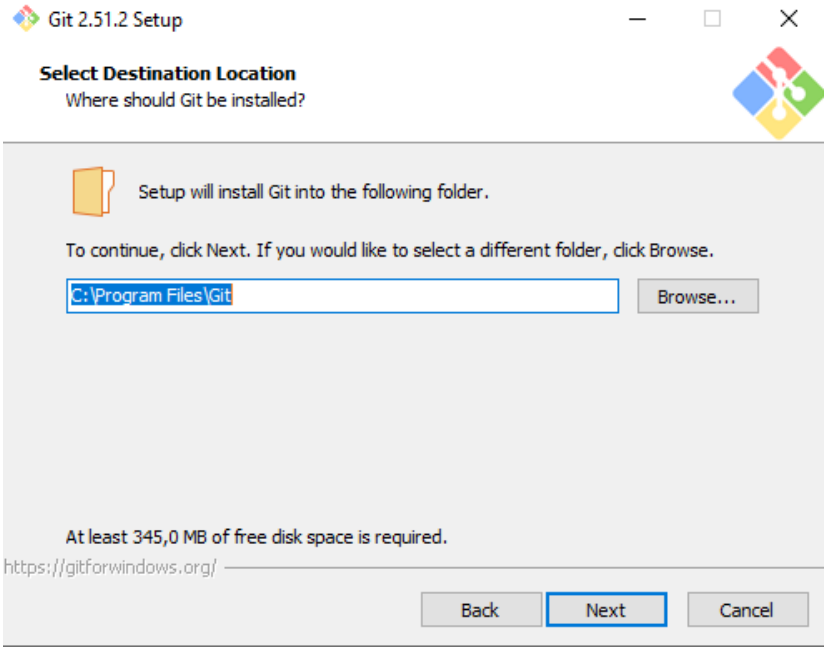
Accedemos a la dirección: https://git-scm.com/ donde descargaremos la última versión de git que para el momento del desarrollo de este documento se encuentra en la 2.51.2.



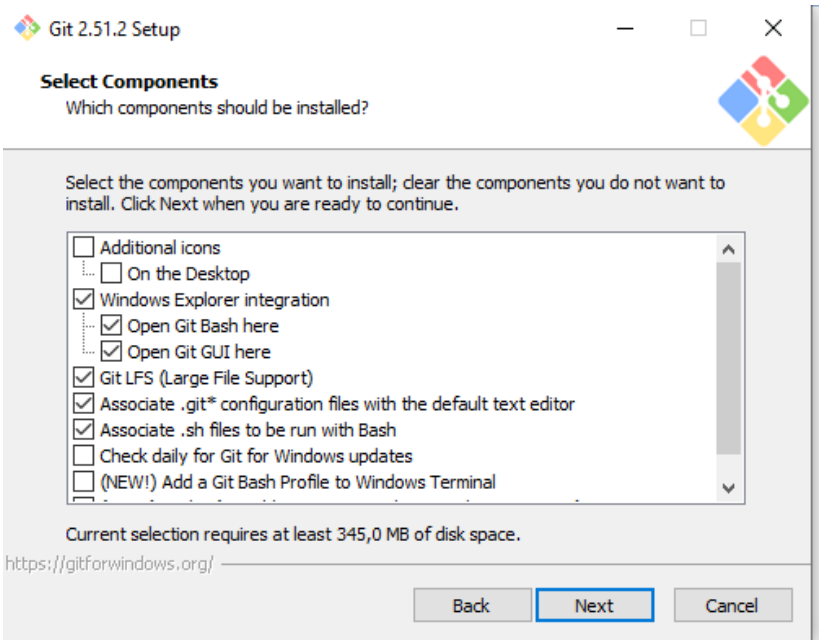



Una vez descargado el archivo, procedemos a instalar, dando luego clic en Next

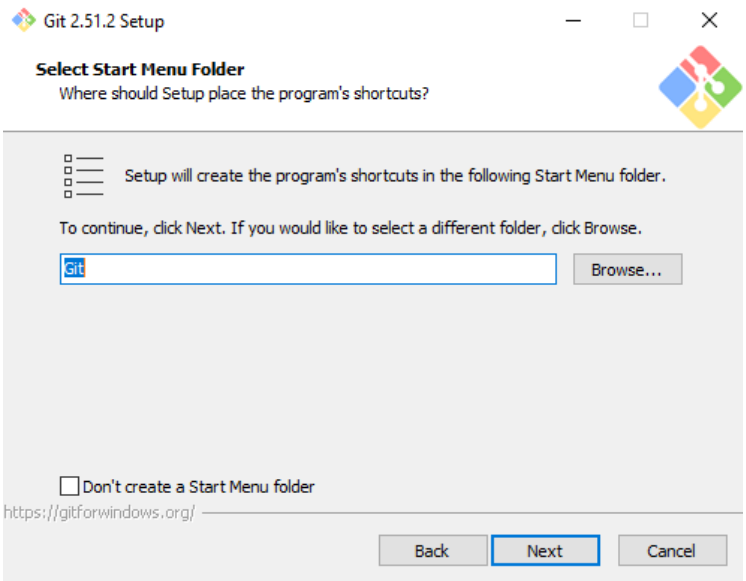
Instalamos en la ruta deseada y luego clic en Next



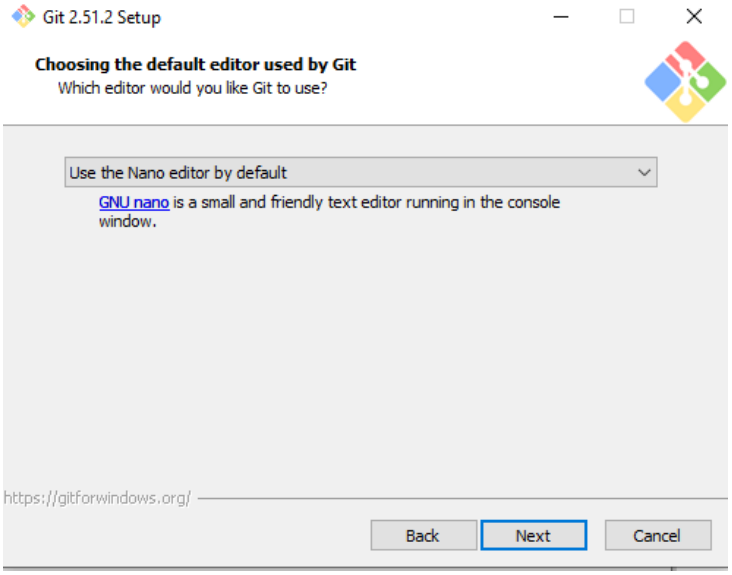

Se nos muestran opciones mediante listas de chequeo a lo quedejaremos las predeterminadas y damos clic en Next



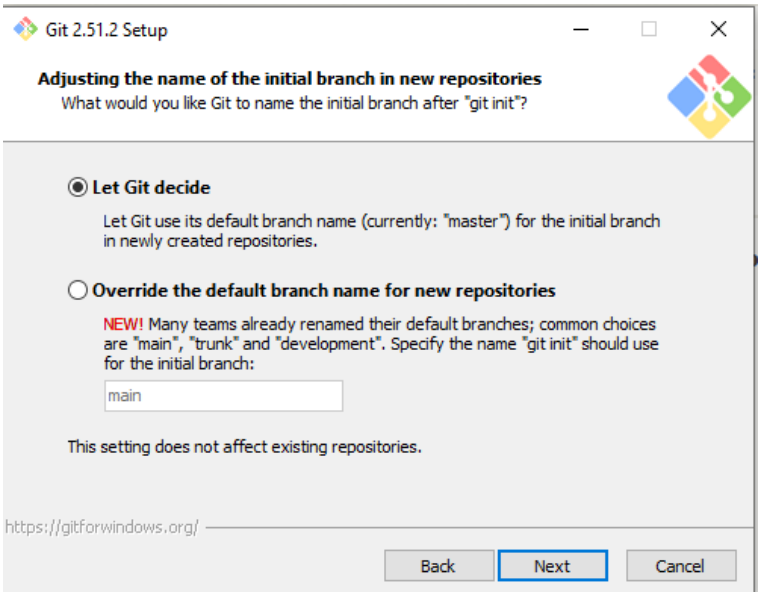
Next


Git 2.51.2 Setup
Select Start Menu Folder
Where should Setup place the program's shortcuts?
Setup will create the program's shortcuts in the following Start Menu folder.
To continue, click Next. If you would like to select a different folder, click Browse.
Git
Browse...
Don't create a Start Menu folder
https://gitforwindows.org/
Back
Next
Cancel

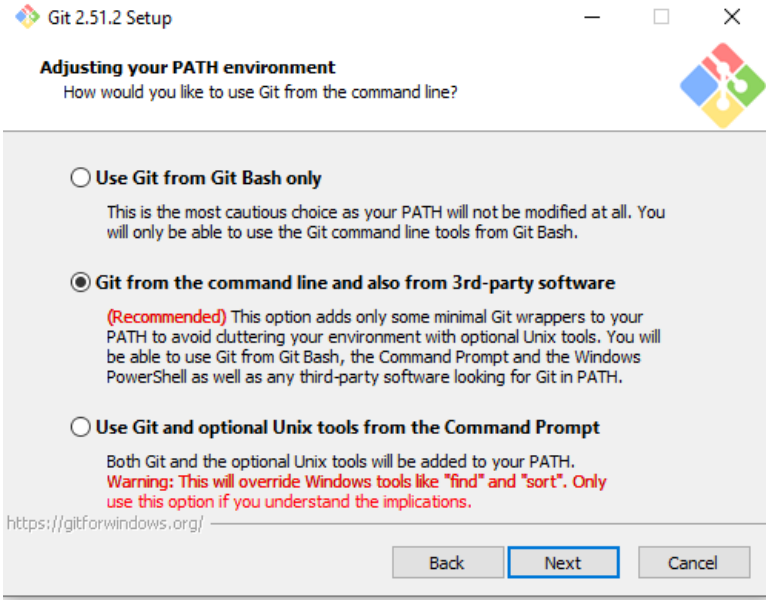
Next


Git 2.51.2 Setup
Choosing the default editor used by Git
Which editor would you like Git to use?
Use the Nano editor by default
GNU nano is a small and friendly text editor running in the console window.
https://gitforwindows.org/
Back
Next
Cancel

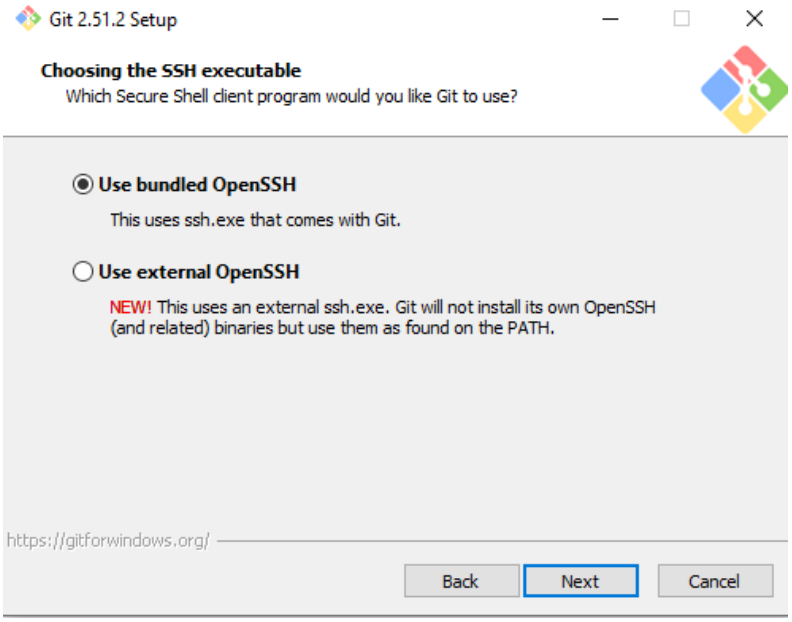
Next


Git 2.51.2 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories.
Override the default branch name for new repositories
NEW! Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
main
This setting does not affect existing repositories.
https://gitforwindows.org/
Back
Next
Cancel

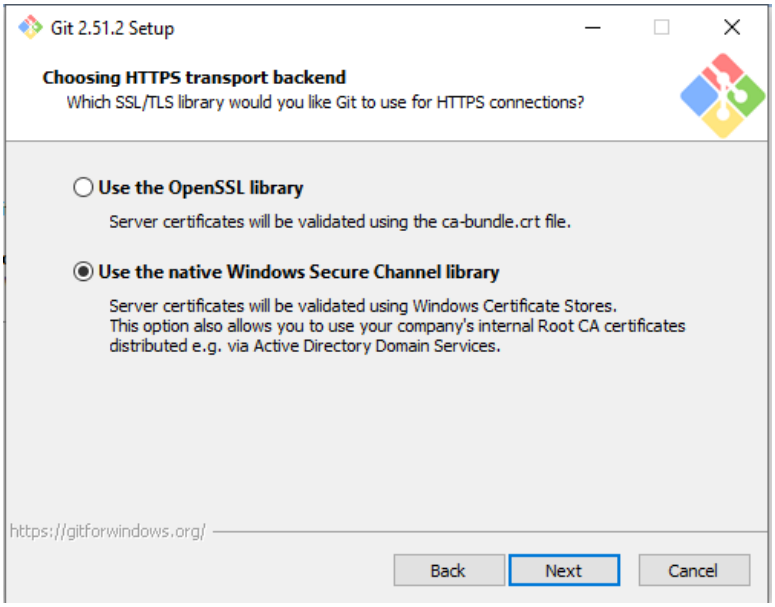
Next


Git 2.51.2 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

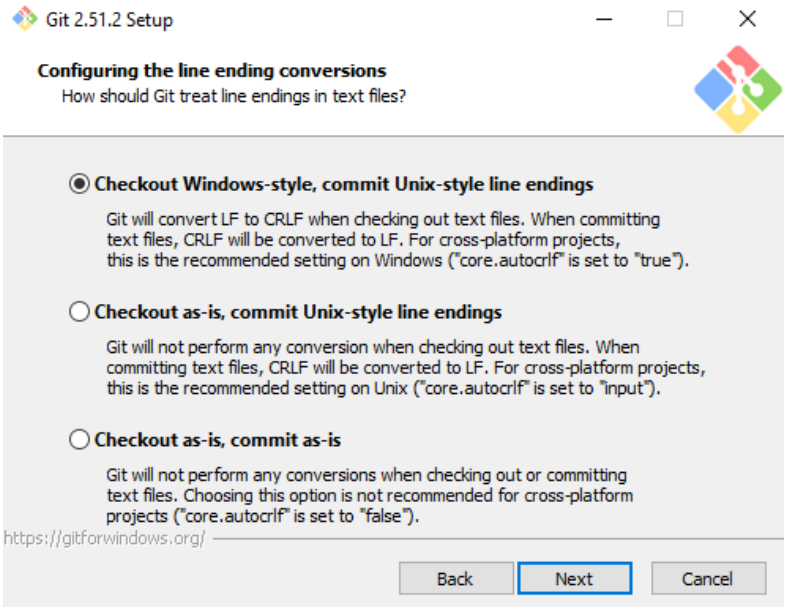
Next


Git 2.51.2 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use external OpenSSH
NEW! This uses an external ssh.exe. Git will not install its own OpenSSH (and related) binaries but use them as found on the PATH.
https://gitforwindows.org/
Back
Next
Cancel

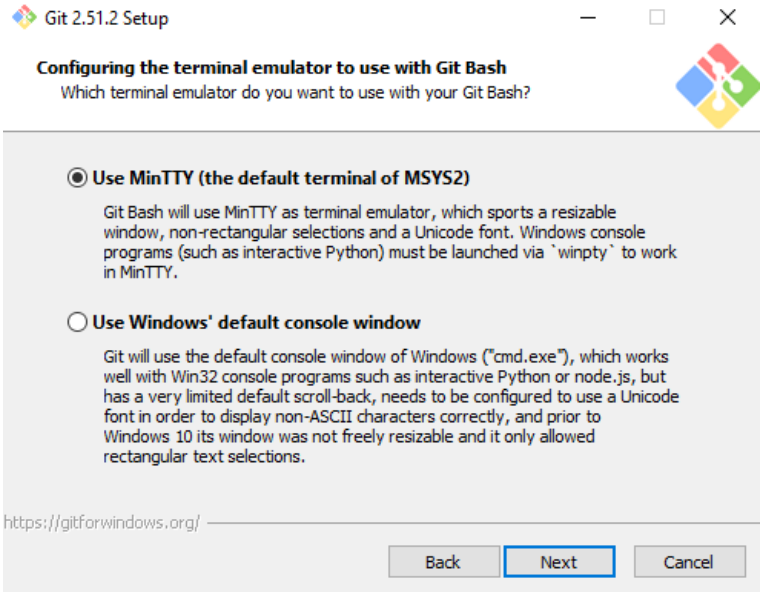
Next


Git 2.51.2 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates distributed e.g. via Active Directory Domain Services.
https://gitforwindows.org/
Back
Next
Cancel

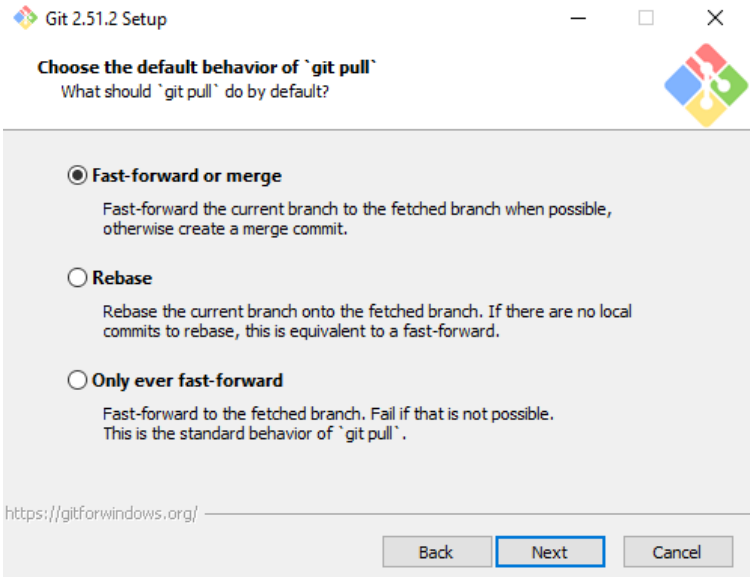
Next


Git 2.51.2 Setup
Configuring the line ending conversions
How should Git treat line endings in text files?
Checkout Windows-style, commit Unix-style line endings
Git will convert LF to CRLF when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Windows ("core.autocrlf" is set to "true").
Checkout as-is, commit Unix-style line endings
Git will not perform any conversion when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Unix ("core.autocrlf" is set to "input").
Checkout as-is, commit as-is
Git will not perform any conversions when checking out or committing text files. Choosing this option is not recommended for cross-platform projects ("core.autocrlf" is set to "false").
https://gitforwindows.org/
Back
Next
Cancel

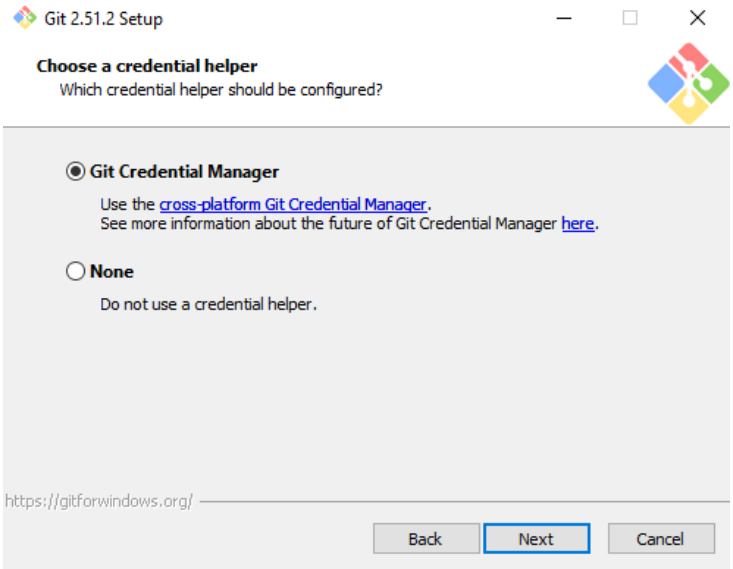
Next


Git 2.51.2 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://gitforwindows.org/
Back
Next
Cancel

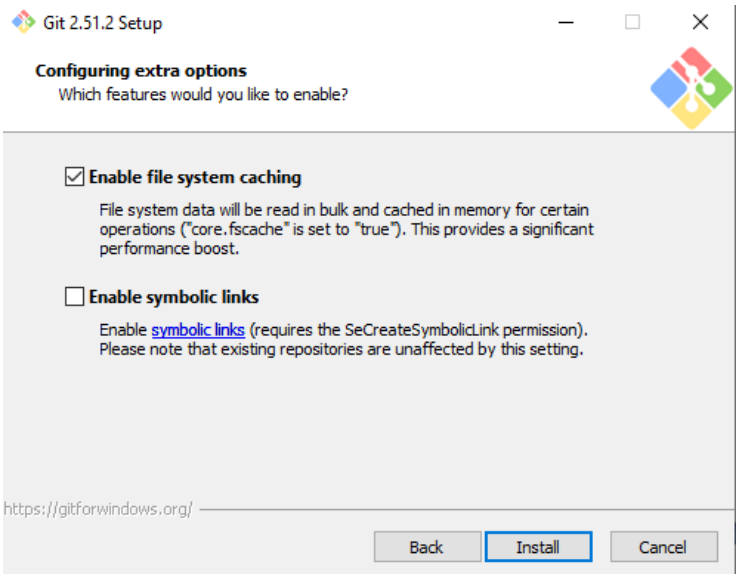
Next


Git 2.51.2 Setup
Choose the default behavior of `git pull`
What should `git pull` do by default?
Fast-forward or merge
Fast-forward the current branch to the fetched branch when possible, otherwise create a merge commit.
Rebase
Rebase the current branch onto the fetched branch. If there are no local commits to rebase, this is equivalent to a fast-forward.
Only ever fast-forward
Fast-forward to the fetched branch. Fail if that is not possible. This is the standard behavior of `git pull`.
https://gitforwindows.org/
Back
Next
Cancel

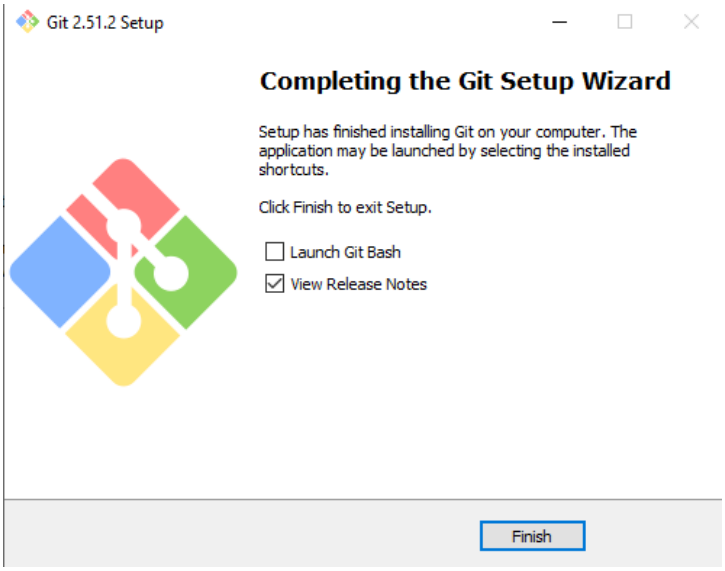
Next


Git 2.51.2 Setup
Choose a credential helper
Which credential helper should be configured?
Git Credential Manager
Use the cross-platform Git Credential Manager.
See more information about the future of Git Credential Manager here.
None
Do not use a credential helper.
https://gitforwindows.org/
Back
Next
Cancel

Install




Finish




Una vez instalado Git este queda integrado con el sistema operatiovo y puede usarse desde cualquier aplicación que haga uso de terminales como bash, power Shell, cmd, Visual studio code, entre otros.

Para verificar la correcta instalación abriremos el cmd y tecleareremos el comando **git --version** si todo ha sido bien instalado deberá mostrarse la versión de Git instalada.

## 3. Ejercicio para pruebas

Para las pruebas de este documento tomaremos de base el siguiente ejercicio: programa básico en Python que convierte una instrucción en texto a una consulta SQL ejecutada mediante SQLAlchemy y SQLite.

El programa realiza las siguientes funcionalidades:

- Crea una base de datos SQLite llamada personas.db.
- Define una tabla personas con id, nombre y edad.
- Recibe comandos en lenguaje natural simple, como:
  agrega persona Juan 25
  muestra todas las personas
  borra persona Juan
- Convierte el texto a una acción SQL usando SQLAlchemy.

**Nota 1:** este programa lo guardaremos en una carpeta llamada**text_to_sql**

**Nota 2:** SQLAlchemy es un conjunto de herramientas SQL y un mapeador objeto-relacional (ORM) que permite vincular clases de Python con tablas de bases de datos relacionales, facilitando una gestión más estructurada y eficiente de los datos.

Para instalarla se debe teclear en la terminal**: pipinstallsqlalchemy**

**Text_to_sql_simple.py**

```
fromsqlalchemyimportcreate_engine,Column,Integer,String,text
fromsqlalchemy.ormimportdeclarative_base,sessionmaker

# --- Configuración básica ---
Base =declarative_base()
engine=create_engine("sqlite:///personas.db",echo=True)
Session=sessionmaker(bind=engine)
session=Session()

# --- Definición del modelo ---
classPersona(Base):
    __tablename__ ="personas"
    id=Column(Integer,primary_key=True)
    nombre =Column(String)
    edad =Column(Integer)
```

```
# Crear la tabla si no existe
Base.metadata.create_all(engine)

# --- Función básica de "Text-to-SQL" ---
definterpretar_texto(texto):
    """
    Interpreta una instrucción en texto simple y devuelve una acción SQL.
    Ejemplo:
    'agrega persona Juan 25'
    'muestra todas las personas'
    """
    palabras =texto.lower().split()

    ifpalabras[0]=="agrega"andpalabras[1]=="persona":
        nombre =palabras[2]
        edad =int(palabras[3])
        nueva =Persona(nombre=nombre,edad=edad)
        session.add(nueva)
        session.commit()
        returnf"Persona '{nombre}' agregada con edad {edad}"

    elif texto =="muestra todas las personas":
        personas =session.query(Persona).all()
        if personas:
            return"\n".join([f"{p.id}. {p.nombre} - {p.edad} años"for p
in personas])
        else:
            return"No hay personas registradas."

    eliftexto.startswith("borra persona"):
        nombre =palabras[2]
        persona =session.query(Persona).filter_by(nombre=nombre).first()
        if persona:
```

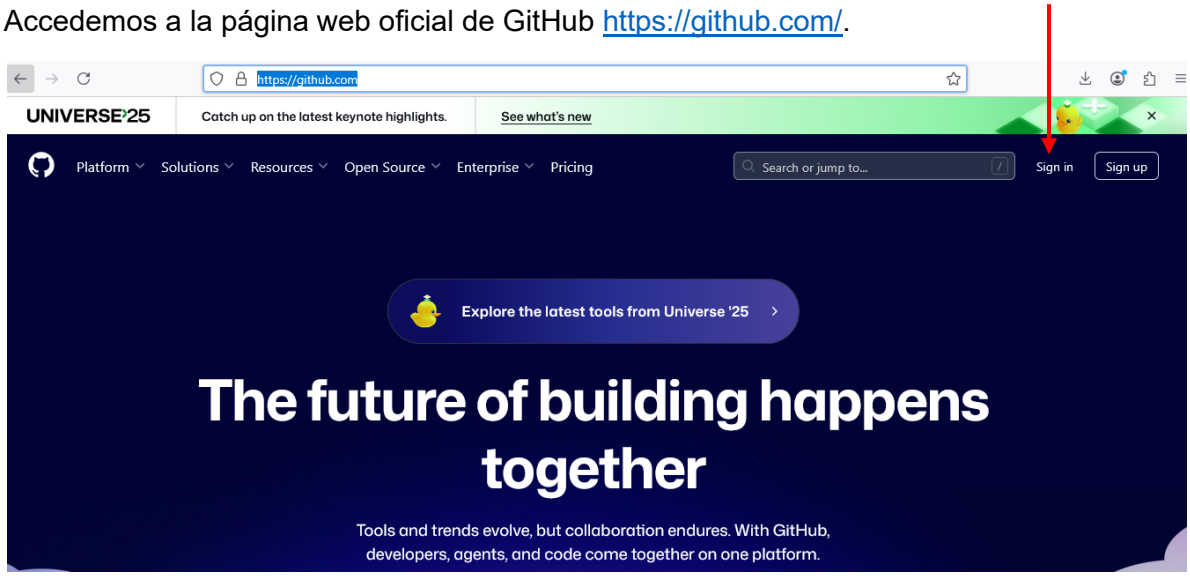
```
            session.delete(persona)
            session.commit()
            returnf"Persona '{nombre}' eliminada."
        else:
            returnf"No se encontró la persona '{nombre}'."

    else:
        return"Instrucción no reconocida."

# --- Ejemplo de uso ---
if__name__=="__main__":
    print("=== Sistema Text-to-SQL Básico ===")
    print("Ejemplos:")
    print("- agrega persona Juan 25")
    print("- muestra todas las personas")
    print("- borra persona Juan")
    print("- salir\n")

    whileTrue:
        comando =input("Escribe tu instrucción: ")
        ifcomando.lower()=="salir":
            break
        resultado =interpretar_texto(comando)
        print(resultado)
```

## 4. Repositorio de GitHub

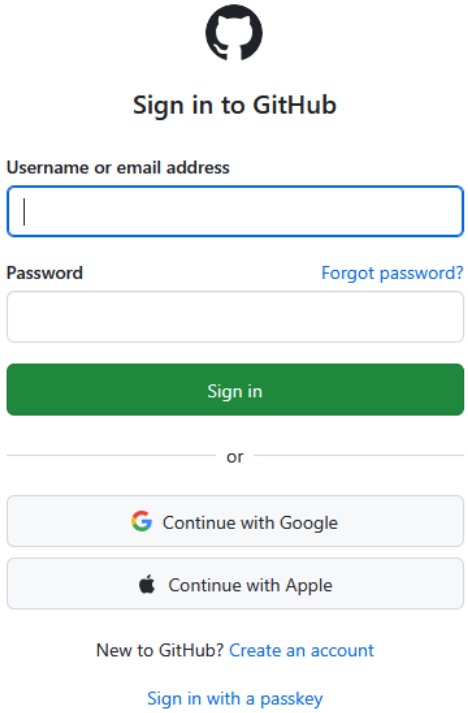
A continuación, crearemos un repositorio donde almacenaremos el programa **Text_to_sql_simple.py**

Para lo cual debemos acceder a una cuenta de GitHub en el caso de no tener una debemos proceder a crearla.

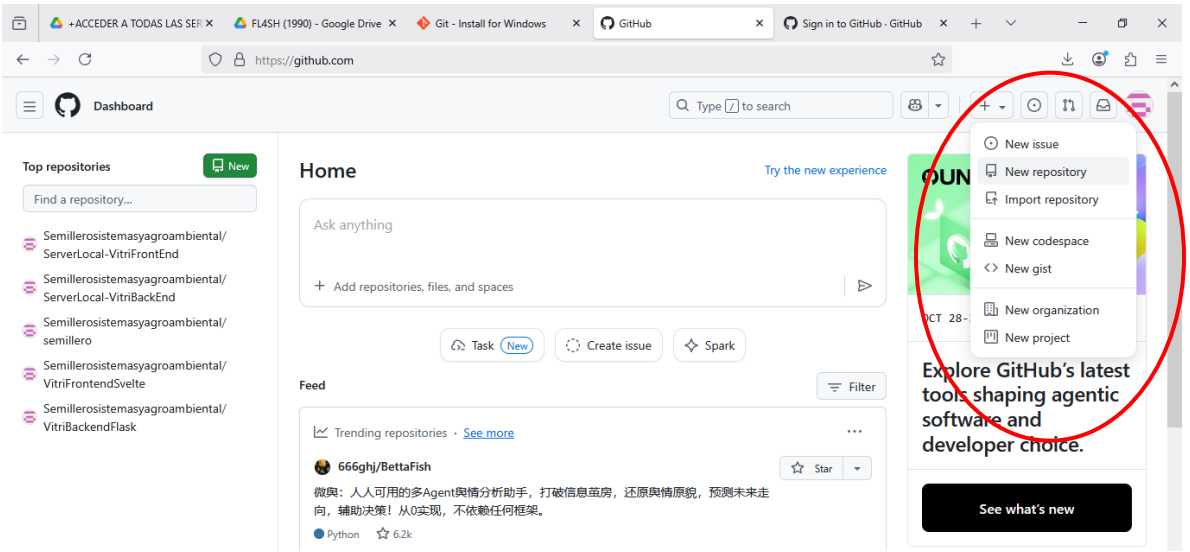
Accedemos a la página web oficial de GitHub https://github.com/.



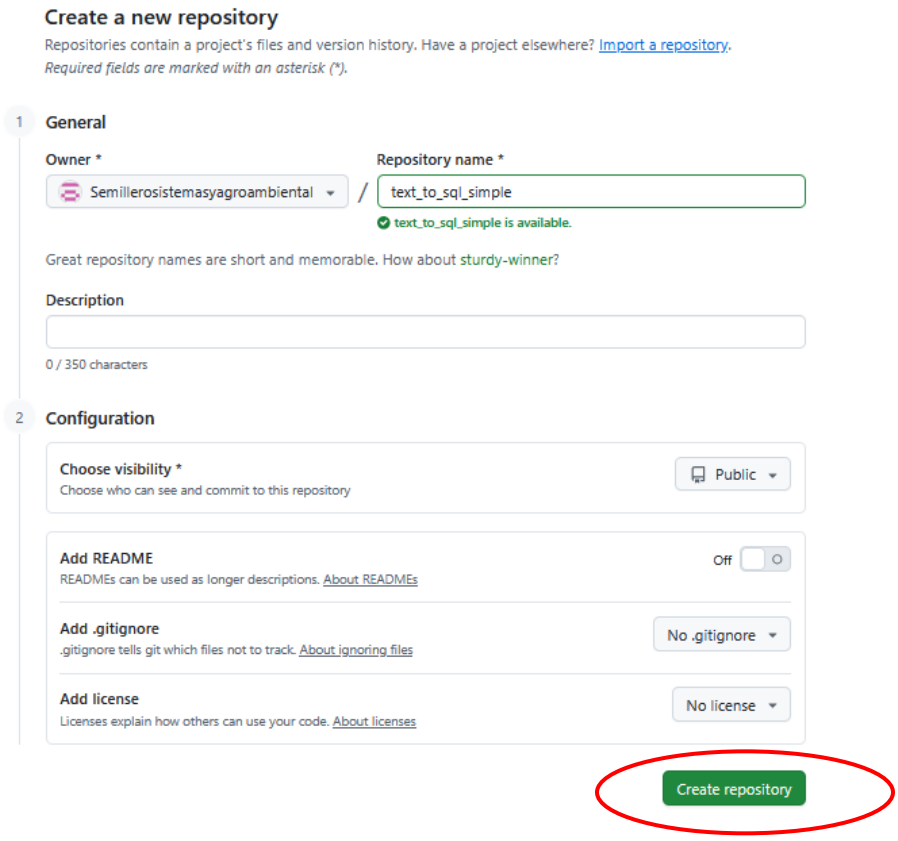

Ingresamos con nuestra cuenta



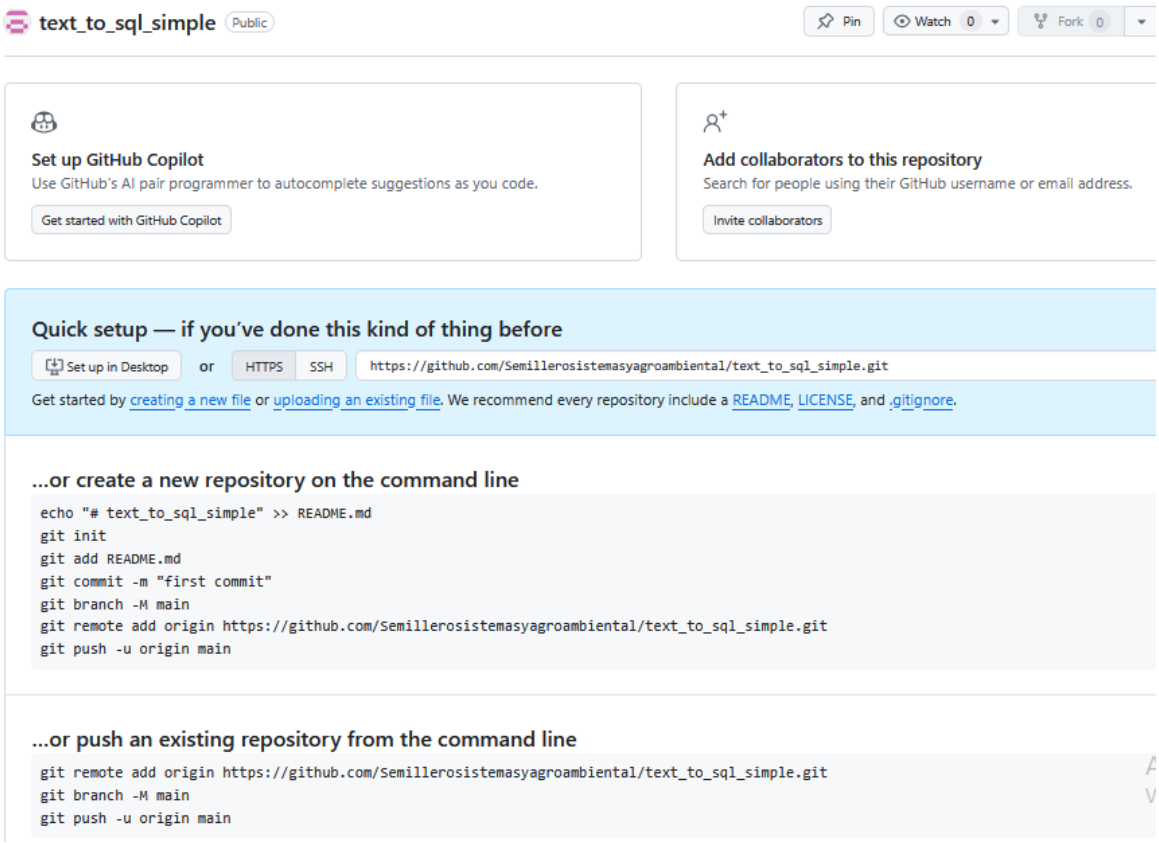
Una vez dentro de nuestra cuenta creamos un nuevo repositorio:




Le daremos por nombre **text_to_sql_simple** o alguno de su preferencia.

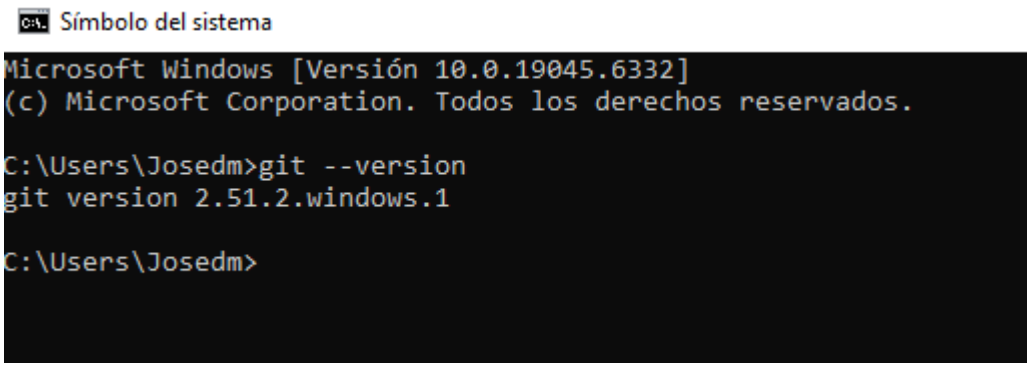


Nos debe aparecer inicialmente el siguiente pantallazo:




Nos dirigimos a una terminal de comandos para cargar nuestro programa, para este caso usaremos el cmd.

Estado actual del cmd:




Nos dirigiremos a la carpeta donde se encuentra el programa que para este caso se encuentra en la unidad **d:** a la cual nos desplazaremos inicialmente y luego por medio del comando cd (change dir) al respectivo directorio como se muestra a continuación:

Símbolo del sistema

```
Microsoft Windows [Versión 10.0.19045.6332]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Josedm>git --version
git version 2.51.2.windows.1

C:\Users\Josedm>d:

D:\>cd text_to_sql

D:\Text_to_sql>
```

Al digitar el comando dir podremos ver los archivos que se encuentran en la carpeta:

```
D:\Text_to_sql>dir
 El volumen de la unidad D es disco
 El número de serie del volumen es: 96B8-1168

 Directorio de D:\Text_to_sql

03/11/2025  12:02 p. m.    <DIR>          .
03/11/2025  12:02 p. m.    <DIR>          ..
03/11/2025  12:02 p. m.                 0 .gitignore
03/11/2025  11:05 a. m.             2.335 text_to_sql_simple.py
               2 archivos          2.335 bytes
               2 dirs  134.789.357.568 bytes libres

D:\Text_to_sql>
```

Podemos observar que se encuentran dos archivos, **text_to_sql_simple.py** que es nuestro programa y **.gitignore** que es un archivo que sirve para listar los archivos y carpetas que no queremos que se carguen al repositorio.

Como es la primera vez que cargaremos en nuestro repositorio debemos configurar nuestro correo y cuenta de GitHub con los siguientes comandos:

**git config --global user.email** [you@example.com](mailto:you@example.com)

**git config --global user.name "Your Name"**

Nota: puede omitirse global para establecer identidad solo en el repositorio de trabajo actual.

Para este caso quedaría:

```
D:\Text_to_sql> git config --global user.email "semilleromsoftlearning@gmail.com"

D:\Text_to_sql>git config --global user.name "semillerosistemasyagroambiental"

D:\Text_to_sql>
```

Nota: en tu caso debes usar el correo y nombre de repositorio que creaste.

Ahora retomando nuestro repositorio creado inicializamos la carpeta con el comando **git init** quien es el encargado de inicializar la carpeta, este proceso solo debe hacerse una vez.

```
D:\Text_to_sql>git init
Initialized empty Git repository in D:/Text_to_sql/.git/
```

Luego tecleamos **git add .** Para que se carguen todos los archivos de la carpeta a excepción de los que se encuentren listados en **.gitignore**

```
D:\Text_to_sql>git add .
```

Continuamos digitando el comando **git commit –m "Primer commit"** que registra los cambios realizados en el código la parte que va en comillas puede tener el nombre que el usuario desee por ejemplo "commit 1", "primer cargue" etc.

```
D:\Text_to_sql>git commit -m "Primer commit"
[master (root-commit) 4c4d154] Primer commit
 2 files changed, 72 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 text_to_sql_simple.py

D:\Text_to_sql>
```

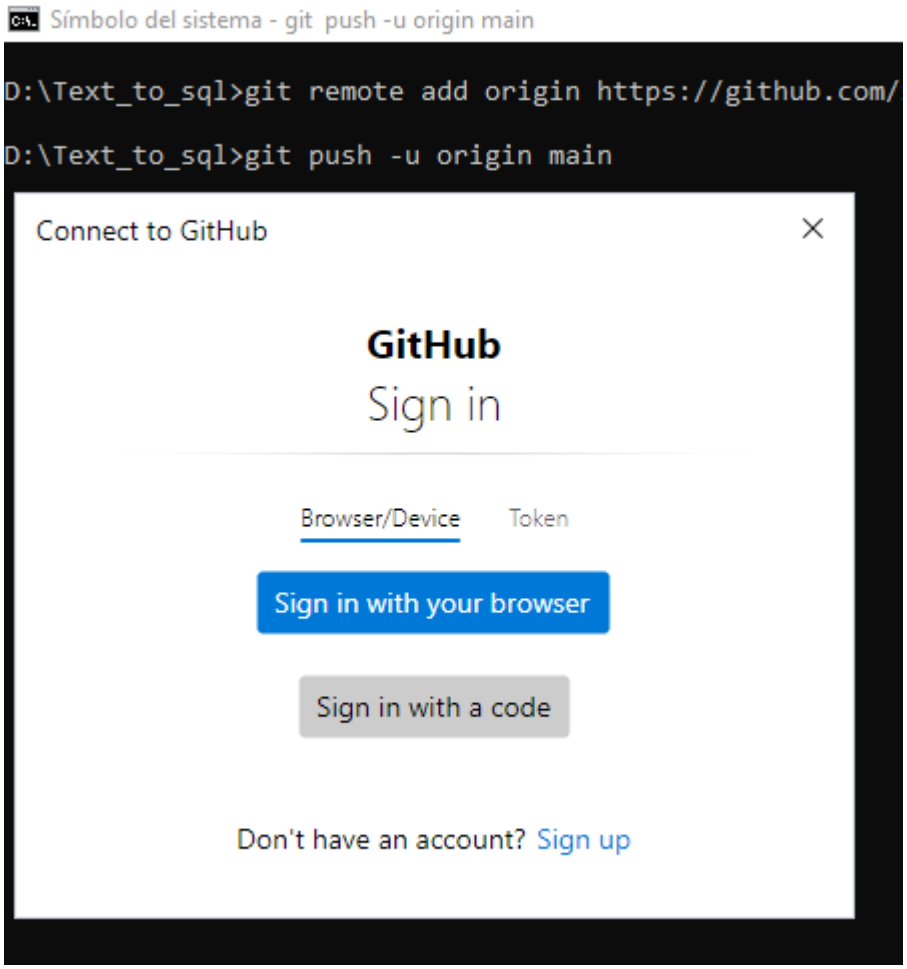
Ahora usamos el siguiente comando para crear la rama principal (solo una vez durante el proceso): **git branch -M main**

```
D:\Text_to_sql>git branch -M main

D:\Text_to_sql>
```

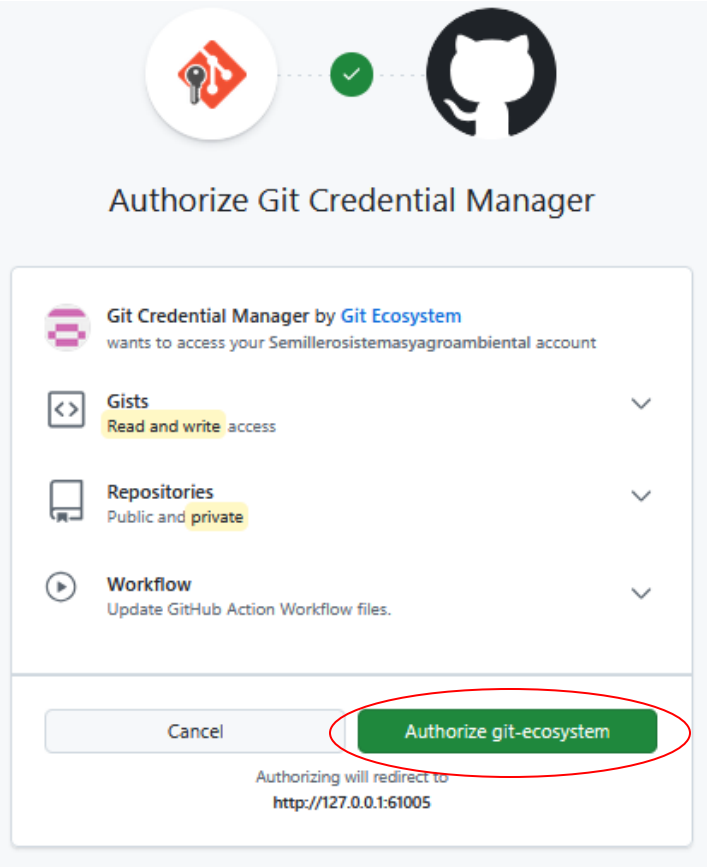
Usamos el comando git remote para llamar el repositorio creado tener en cuenta que solo se hace una vez durante el proceso: **git remote add origin**
**https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git**

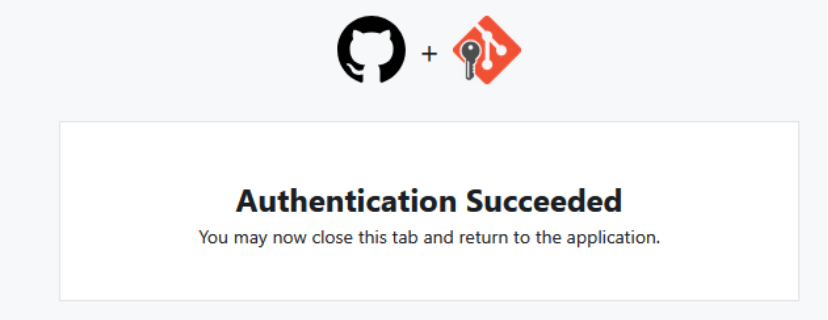
```
D:\Text_to_sql>git remote add origin https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git

D:\Text_to_sql>
```

Por último, para este primer cargue: **git push -u origin main**
Aparecerá un pantallazo que nos solicitará ingresar a nuestra cuenta de GitHub




Al ingresar por la primera opción nos solicitara autorización

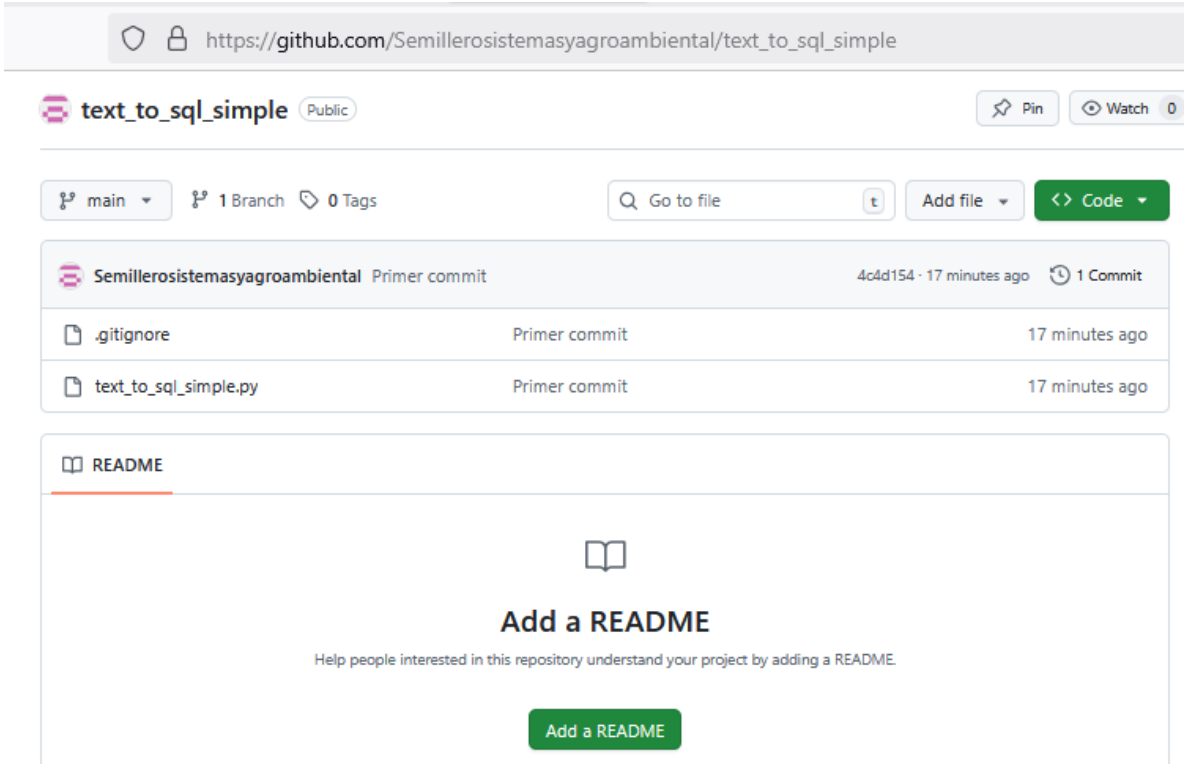



Símbolo del sistema

```
D:\Text_to_sql>git remote add origin https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git

D:\Text_to_sql>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.18 KiB | 602.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

D:\Text_to_sql>
```

Al actualizar en el explorador nuestro repositorio ya debe aparecer cargado el programa.



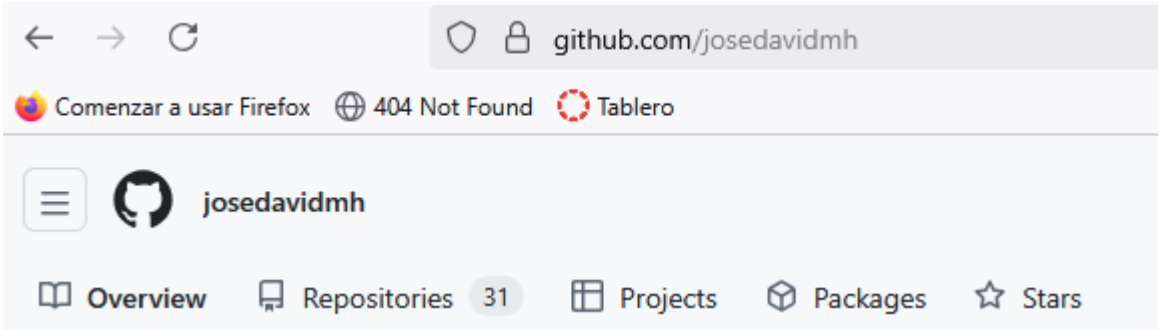

## 5. Trabajo colaborativo con fork

Se tomará de base el ejemplo anterior, pero para este caso necesitaremos de otra cuenta de GitHub.

Para tener un mayor contexto es recomendable ver el video relacionado a continuación:

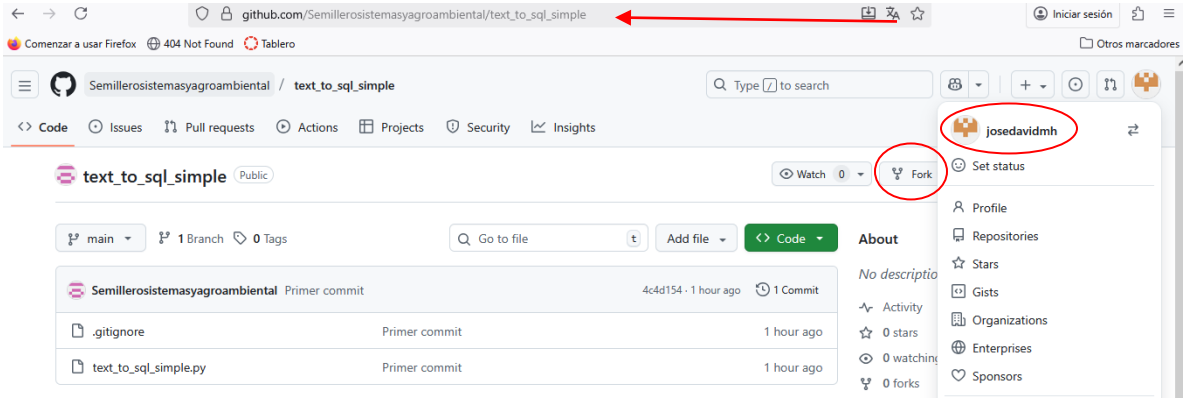
[https://www.youtube.com/watch?v=4YlHQAETkPs](https://www.youtube.com/watch?v=4YlHQAETkPs)

Nos logueamos en la segunda cuenta de GitHub que para este caso es la siguiente:



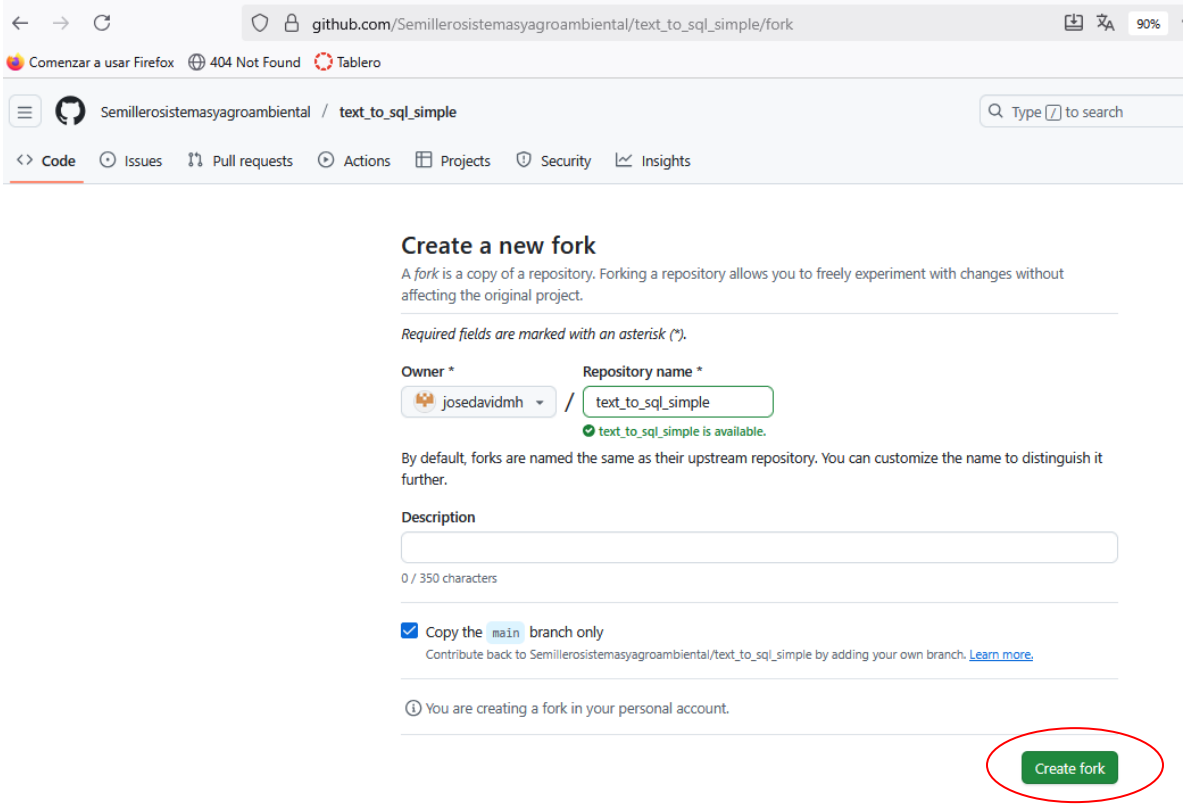

Una vez logueado tomamos el enlace al repositorio de código anterior y lo pegamos en el explorador para poder hacer el fork:

https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple

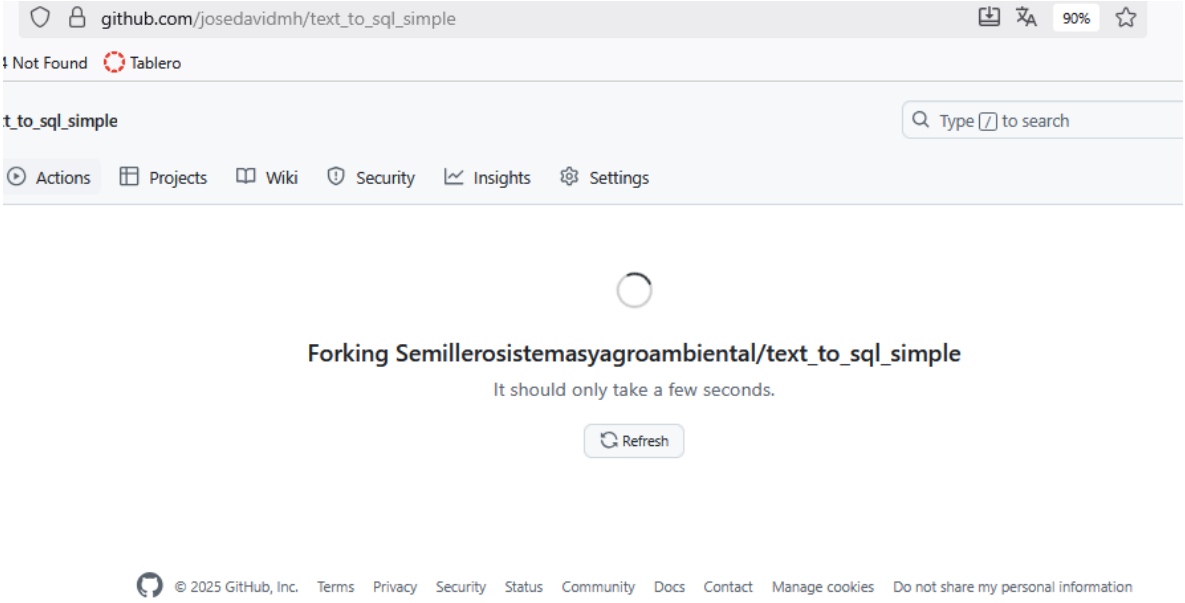


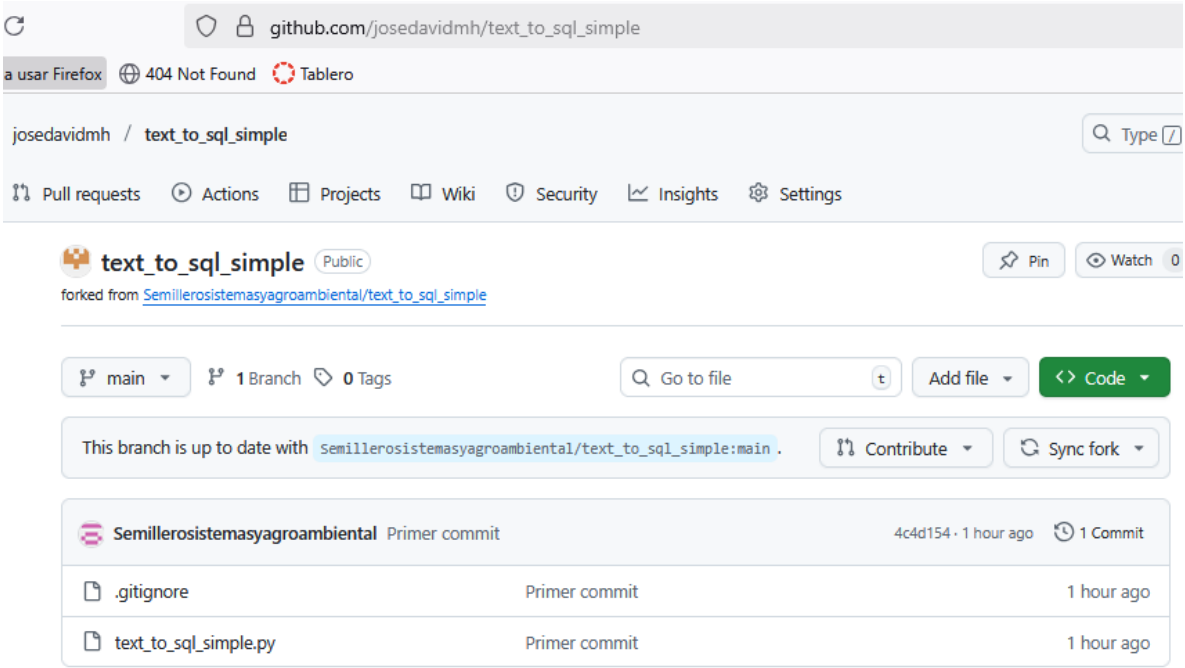

Podemos observar que la cuenta activa es **josedavidmh** que corresponde a la segunda cuenta (En tu caso reemplazas por tu segunda cuenta o la de un compañero).

Al dar clic en fork nos sale el siguiente pantallazo:

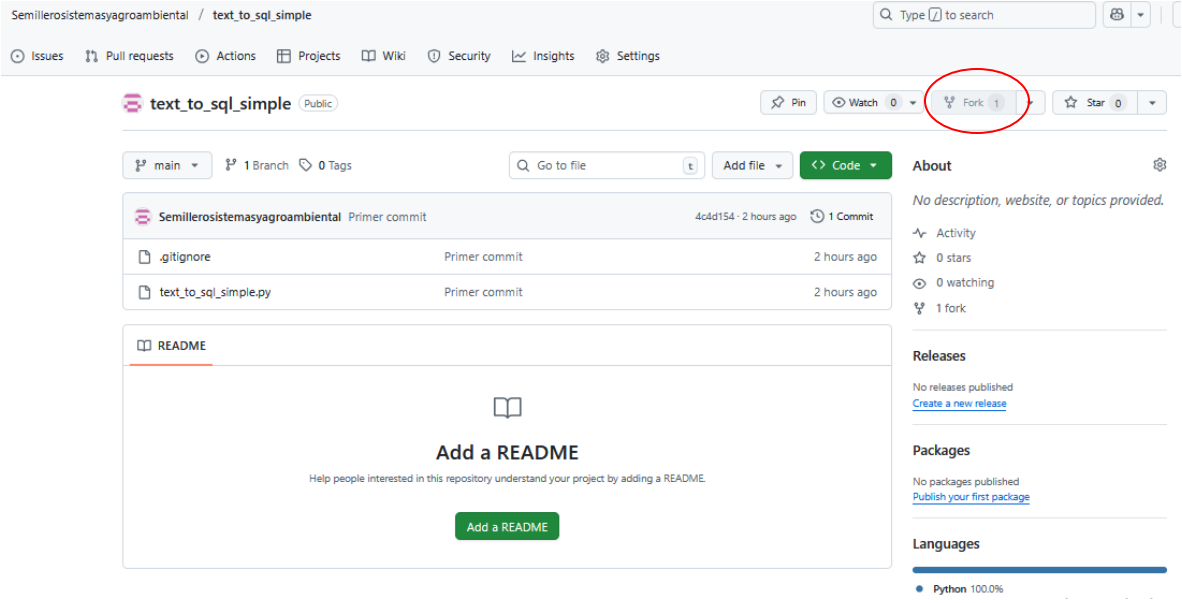



Al hacer clic en fork se enlaza el proyecto con nuestro repositorio.



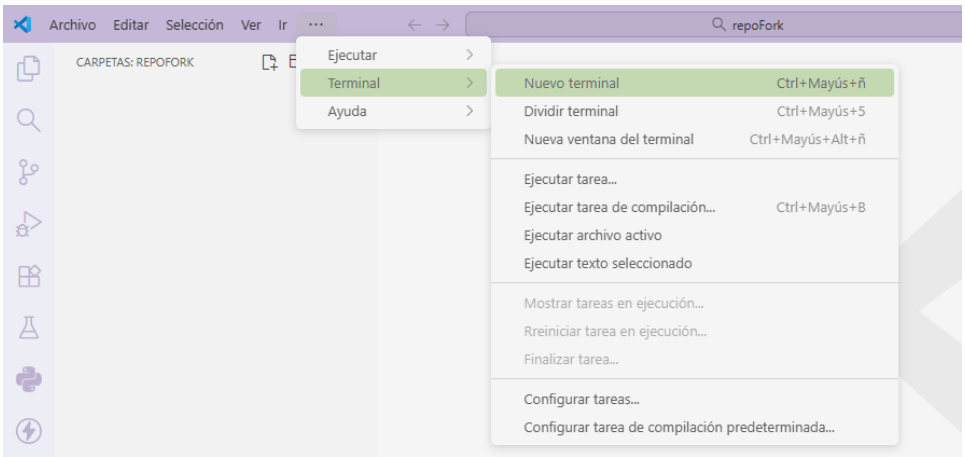

Al revisar en el repositorio original podemos evidenciar que en botón fork aparece un 1 que hace referencia al numero de personas que se han conectado para colaborar con el proyecto.



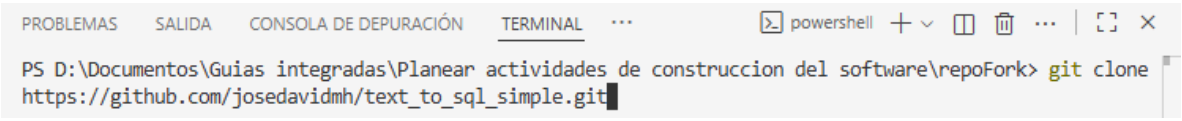

Una vez enlazado el repositorio a la cuenta B procedemos a clonar el repositorio enlazado, Para ello usaremos una carpeta llamada repoFork o un nombre de su preferencia.

Nota: las acciones de la cuenta B, se harán en Visual Studio Code mientras que las de la cuenta A se seguirán realizando en el CMD.

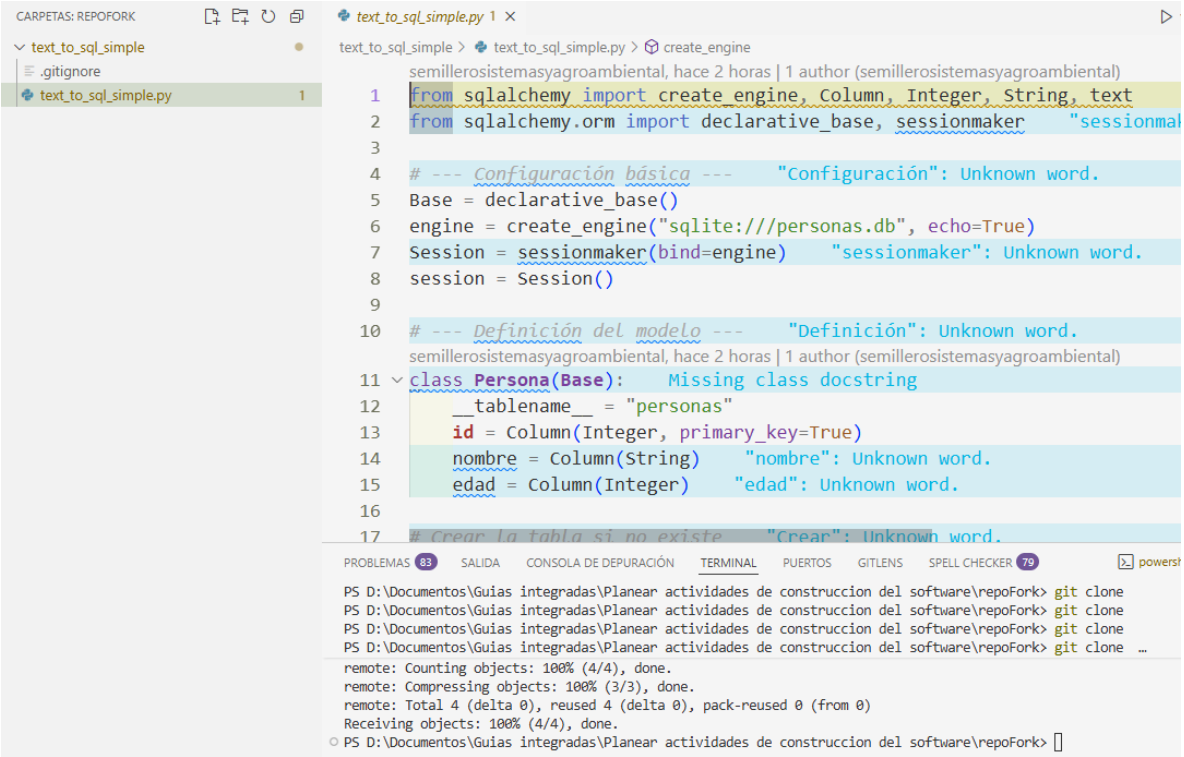
En vs code abrimos una nueva terminal.




En la terminal tecleamos el repositorio forkeado:




Para su caso debe cambiar el enlace de repositorio por el suyo propio.




Una vez se encuentre la carpeta clonada el usuario de la cuenta B puede realizar las modificaciones al código.

Código corregido bajo estándar PEP8:

en este se hicieron ajustes como

- Importación limpia
- Dos líneas en blanco antes y después de definiciones de clases y funciones
- Eliminación de else redundante
- Líneas de máximo 79 caracteres.

Nota: para efectos de practica de este ejercicio puedes hacer los ajustes que consideres necesario.

A continuación, tecleamos en su orden los siguientes comandos:

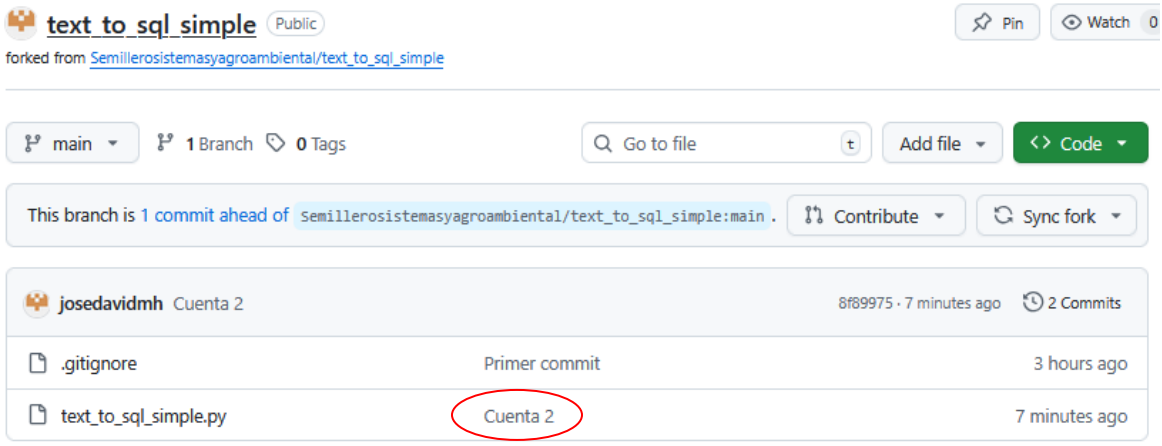
git add .

git commit -m **“cuenta 2”**

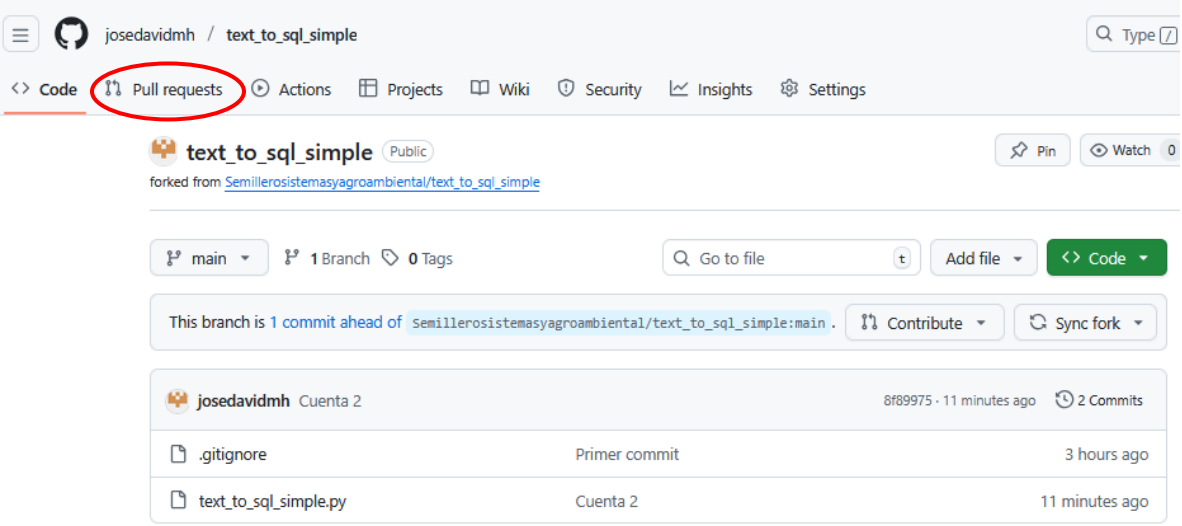
git push -u origin main

```
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple> git add .
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple> git commit -m  "Cuenta 2"
[main 8f89975] Cuenta 2
 1 file changed, 16 insertions(+), 10 deletions(-)
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 394 bytes | 197.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/josedavidmh/text_to_sql_simple.git
   4c4d154..8f89975  main -> main
branch 'main' set up to track 'origin/main'.
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple>
```

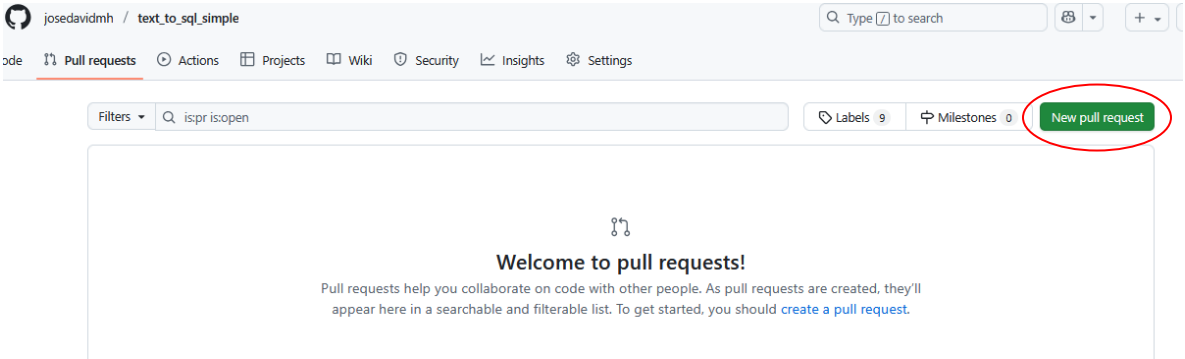
En el repositorio de la cuenta 2 se nos debe reflejar de la siguiente manera:



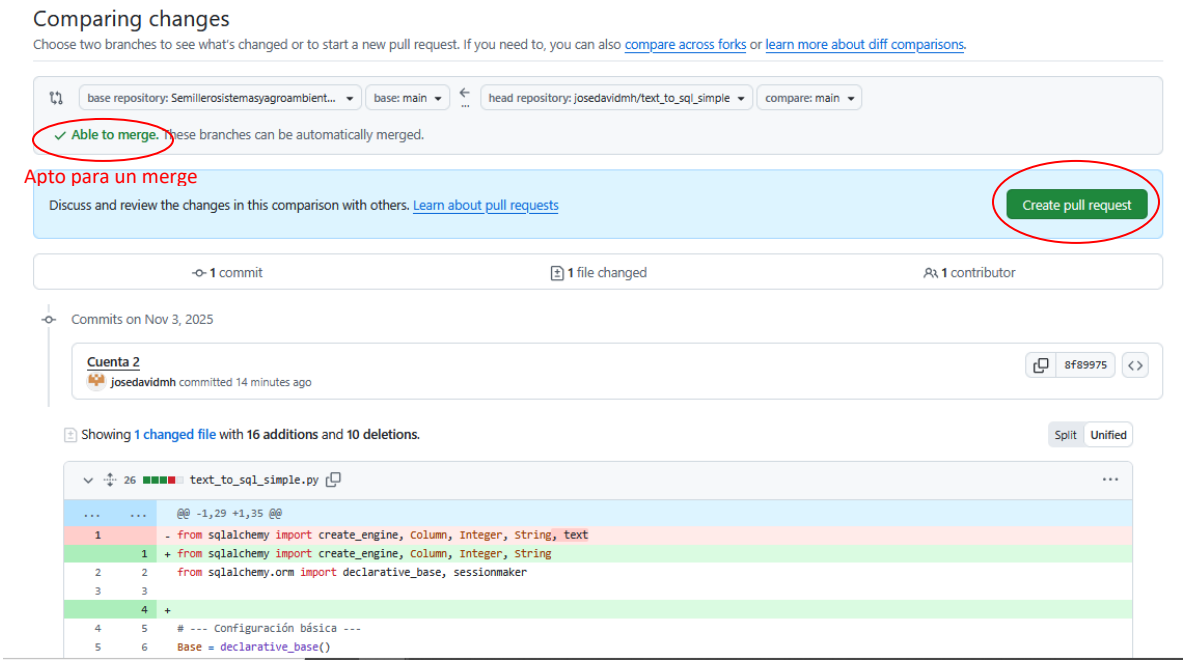

Hasta este punto solo hemos subido al repositorio el programa clonado para la cuenta 2. Ahora haremos un **pull request** para que el usuario de la cuenta original se entere de los cambios que se han realizado y decida si quiere aplicarlos o no.



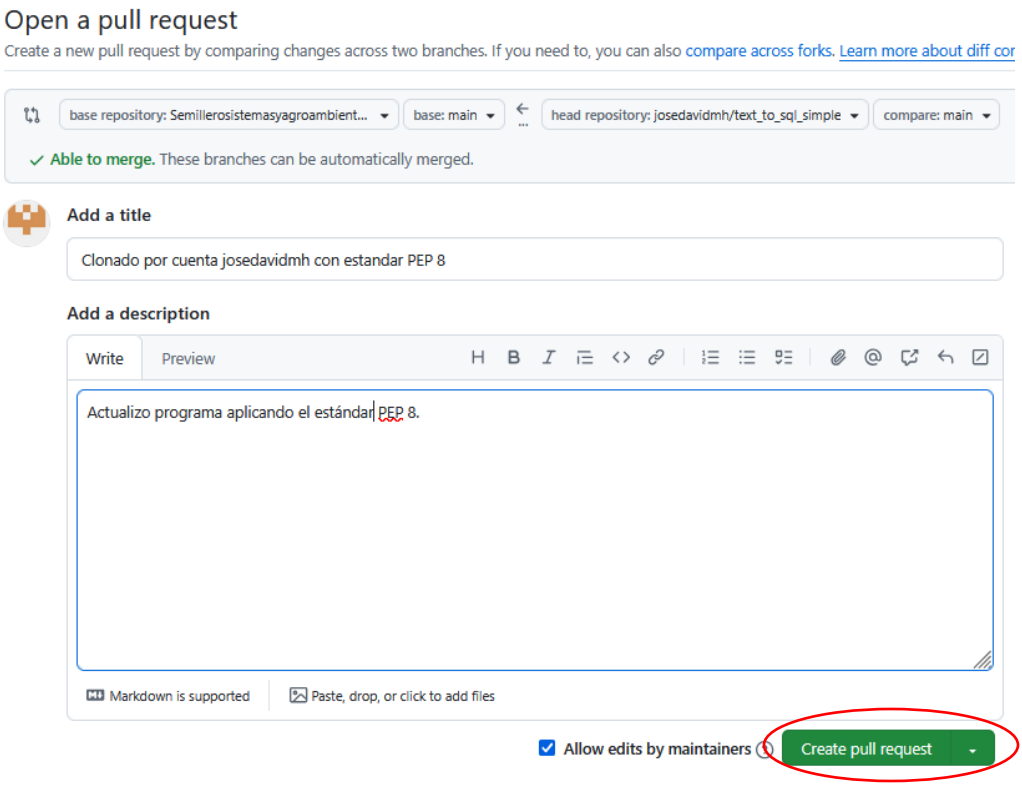
New pull request



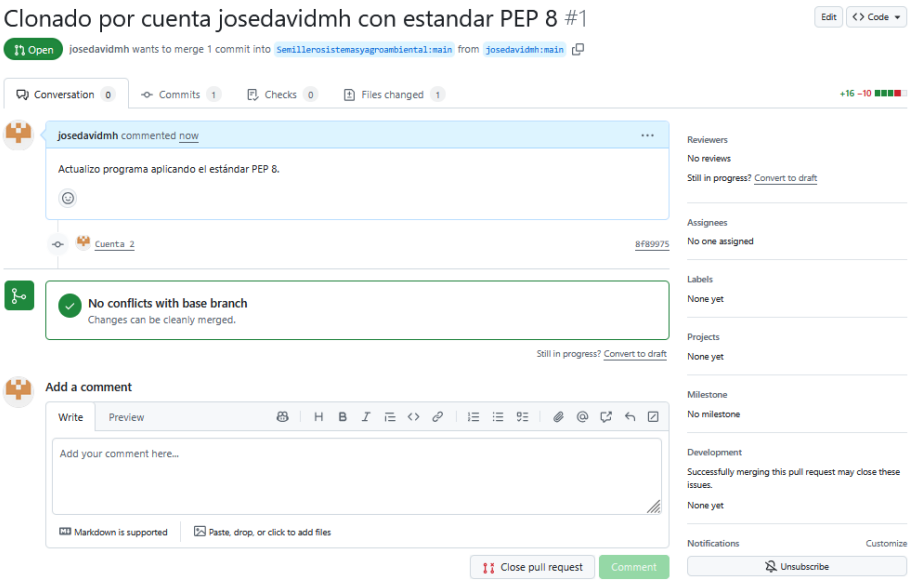

A continuación, se nos muestra un comparativo entre como estaba el código y como se encuentra ahora por medio de un **+** para lo adicionado y un **–** para lo suprimido. Luego damos clic en **Create pull request.**



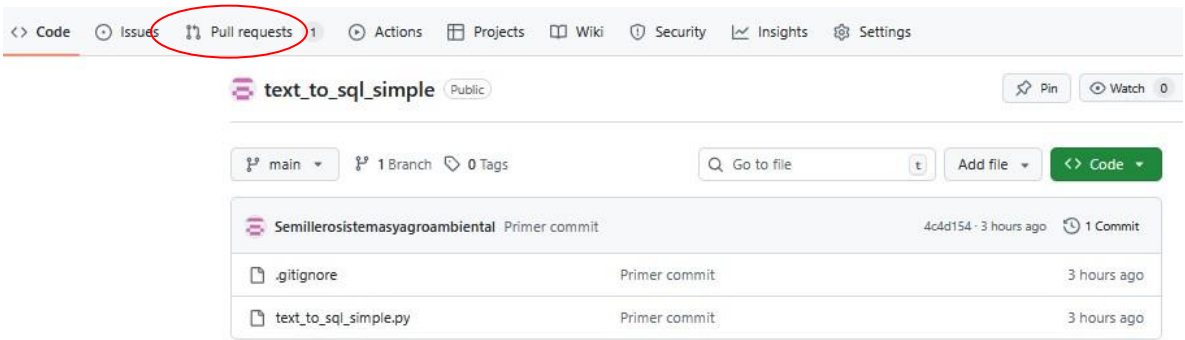

Colocamos un título y una descripción para el **pull request** y para finalizar damos click en el botón **Create pull request.**




Nos aparece el siguiente pantallazo del cual podemos seguir comentando

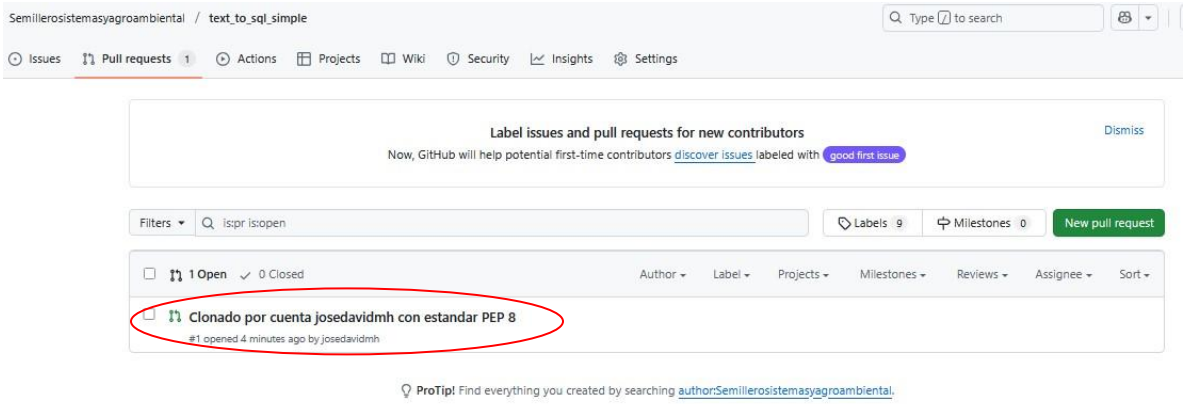



Desde el perfil del creador del programa original aparecerá el siguiente pantallazo

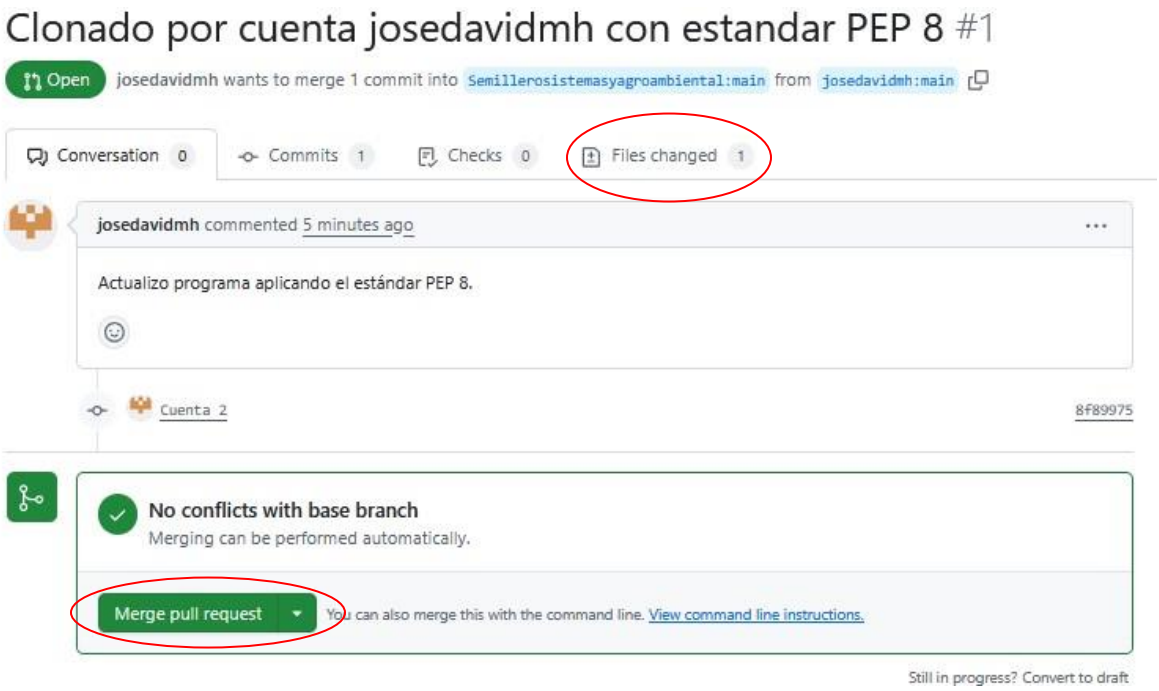



Como podemos ver se conserva la información original con la diferencia que en el botón **Pull request** aparece un 1 indicando que se han hecho cambios en el código por parte de un colaborador.

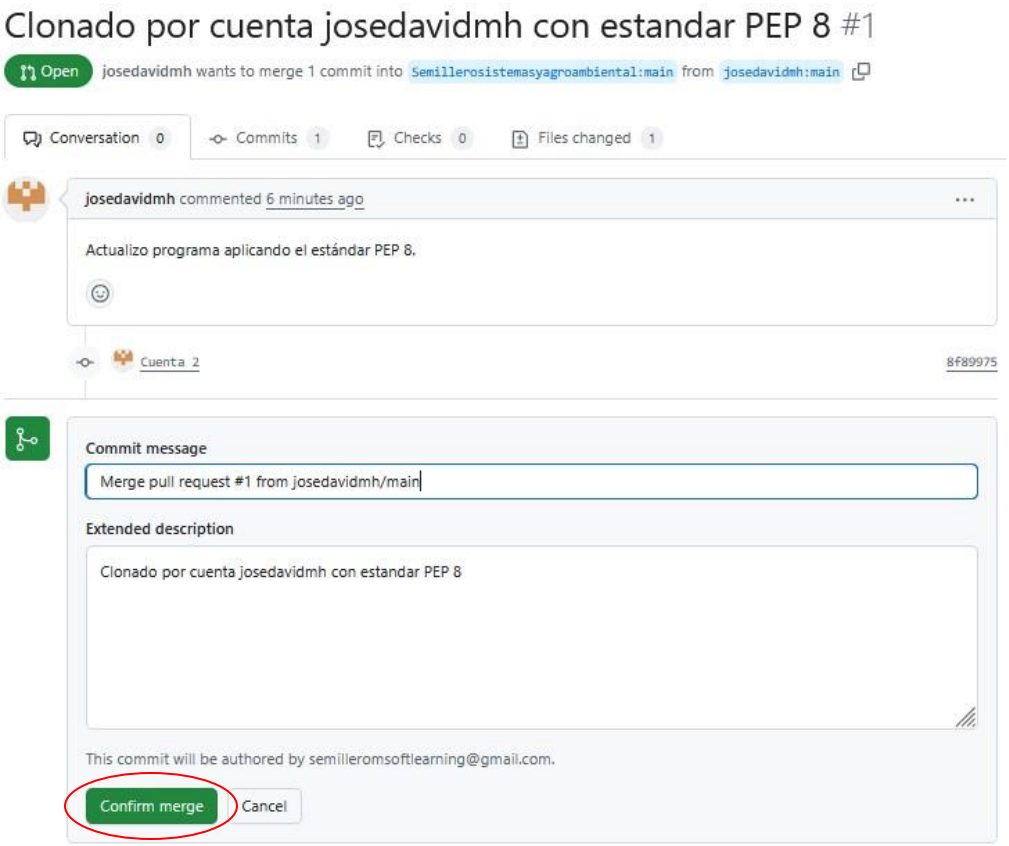
Al darle clic podemos ver un pantallazo mostrándonos el pull request aplicado por la segunda cuenta al cual podemos darle clic para revisar su contenido.



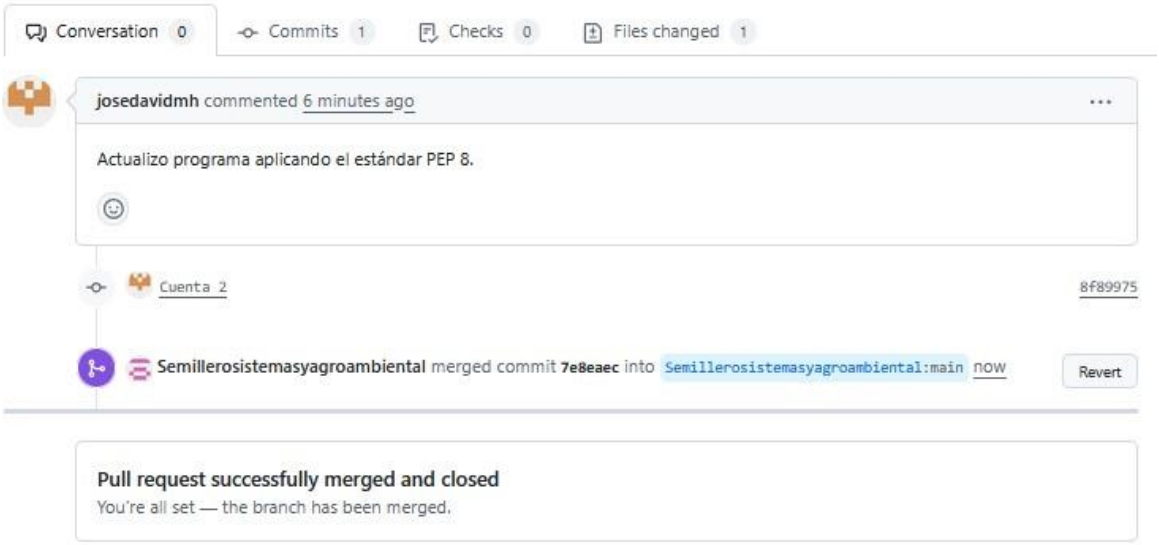

En esta tenemos dos opciones clave **Files changed** para revisar, aprobar o rechazar los cambios y **Mergue pull request** para aprobar directamente los cambios.



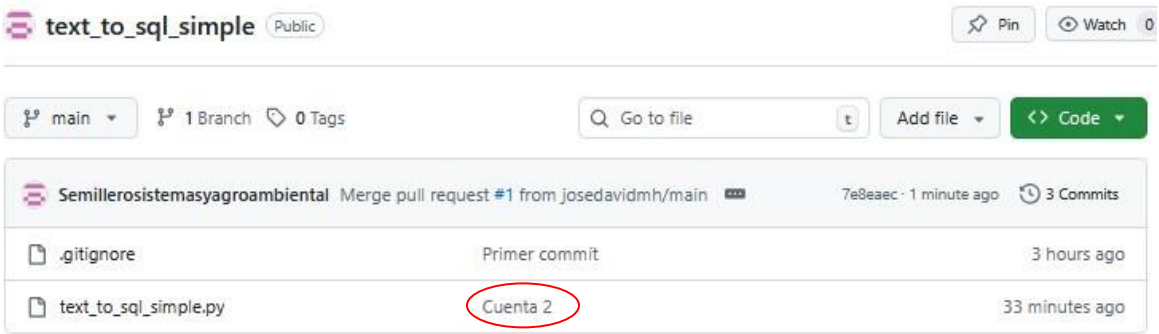

Al dar clic en **Merge pull request** nos aparece un nuevo pantallazo para confirmar.



A continuación, aparecerán los cambios confirmados




Al revisar el código en la cuenta original se deben ver reflejados los cambios




Ahora para reflejar esos cambios en la carpeta local del usuario original hay que hacer un pull con la dirección del repositorio.

```
D:\text_to_sql_simple>git pull https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git
```

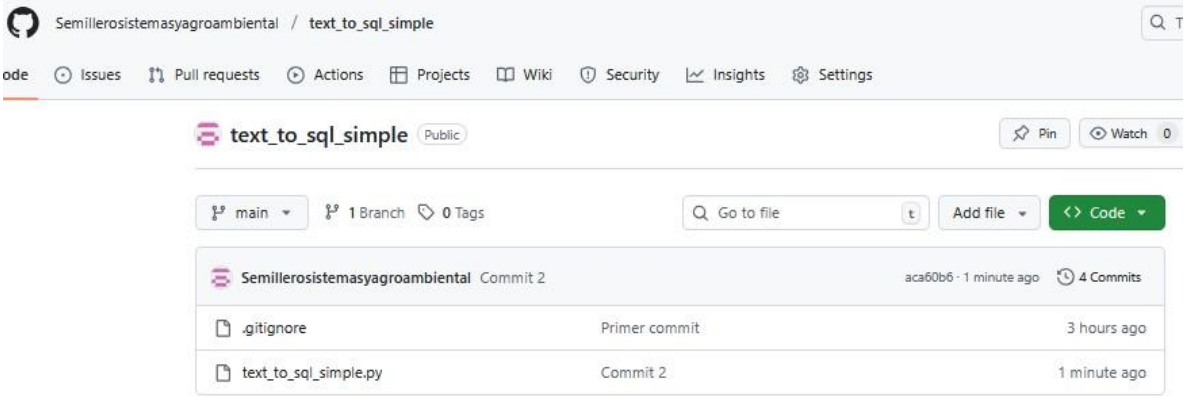
De esta manera si queremos hacer un cambio desde el repositorio original podemos hacerlo manualmente, por ejemplo, agregar al código un comentario

```
1  # Comentario de prueba
2  from sqlalchemy import create_engine, Column, Integer, String
3  from sqlalchemy.orm import declarative_base, sessionmaker
4
```

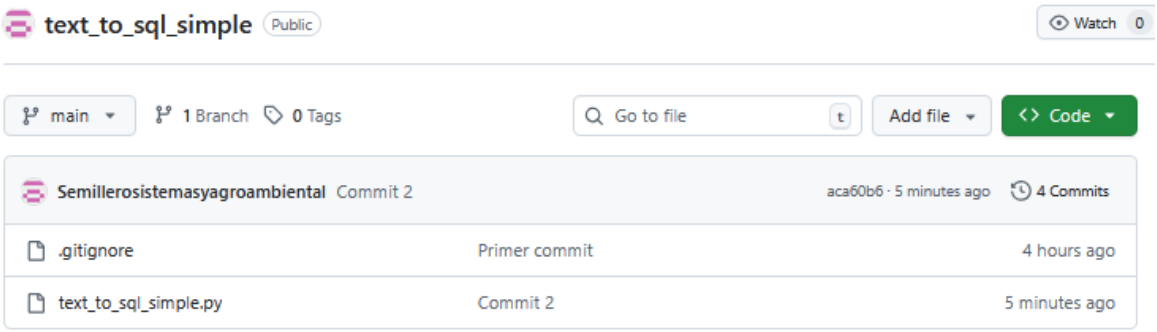
Se sube al repositorio

```
D:\text_to_sql_simple>git add .

D:\text_to_sql_simple>git commit -m "Commit 2"
[main aca60b6] Commit 2
 1 file changed, 1 insertion(+)

D:\text_to_sql_simple>git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 359 bytes | 179.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple.git
   7e8eaec..aca60b6  main -> main
branch 'main' set up to track 'origin/main'.

D:\text_to_sql_simple>
```

El repositorio original ahora debe verse de la siguiente manera




Si nos vamos a la cuenta 2 debe verse también de esta manera



Para actualizar en la carpeta local de la cuenta 2 aplicamos igualmente un **git pull** con el link del repositorio.

```
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple> git pull https://github.com/
Semillerosistemasyagroambiental/text_to_sql_simple.git
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1.23 KiB | 4.00 KiB/s, done.
From https://github.com/Semillerosistemasyagroambiental/text_to_sql_simple
 * branch            HEAD       -> FETCH_HEAD
Updating 8f89975..aca60b6
Fast-forward
 text_to_sql_simple.py | 1 +
 1 file changed, 1 insertion(+)
PS D:\Documentos\Guias integradas\Planear actividades de construccion del software\repoFork\text_to_sql_simple>
```

## 6. Referencias

*Bifurcar un repositorio - Documentación de GitHub*. (s. f.). GitHub Docs. https://docs-github-com.translate.goog/es/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true

Git - seguimiento básico. (s. f.). https://git-scm.com/book/es/v2/Ap%C3%A9ndice-C:-Comandos-de-Git-Seguimiento-B%C3%A1sico

*PEP 8 – Style Guide for Python Code | peps.python.org*. (s. f.). Python Enhancement Proposals (PEPs). https://peps-python-org.translate.goog/pep-0008/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Pildorasinformaticas. (2020, 26 noviembre). Curso git. Fork. vídeo 10 [Vídeo]. YouTube. https://www.youtube.com/watch?v=4YIHQAETkPs

**Control del documento**

| | **Nombre** | **Cargo** | **Dependencia** | **Fecha** |
|---|---|---|---|---|
| **Autor (es)** | Jose David Montesino Hoyos | Instructor | Centro Biotecnológico del caribe | Noviembre de 2025 |

**Control de cambios**

| | **Nombre** | **Cargo** | **Dependencia** | **Fecha** | **Razón del Cambio** |
|---|---|---|---|---|---|
| **Autor (es)** | | | | | |